

Lab 4: Groupy - A Group Membership Service

Sevastian Bajenov

September 29, 2026

1 Introduction

In this laboratory work, my main goal was to explore one of the coordination and agreement use cases - group communication. The final deliverable of this exploration was the atomic multicast implemented within the group membership service (GMS). By using the prerequisites (application layer code and the GUI) I managed to observe different versions of the GMS, starting with a simple ordered multicast and concluding with an atomic multicast within the system.

2 Main problems and solutions

In order to understand how the system works it is necessary to know that each node can be broken down into two layers: application and group communication. The analysis and findings presented next consider the three versions of the GMS starting from the most primitive.

gms1 introduced basic logic for leader and slave nodes. The slaves were resent all of the commands from their application layer (Master process) to the leader node and at the same time delivered the received messages and view updates from the leader to the application layer. By view I mean the current participants of the group. *gms1* had one major flaw - it did not allow the nodes to select a new leader after crash and therefore failure of the leader meant that the system froze.

gms2 introduced the concept of election, that is in case a leader died the first node to receive a 'DOWN' signal selects the first node from the group or proclaims itself as the new leader. Each slave continuously monitors the leader using *erlang:monitor/2* method. However, this approach had its own flaw which consisted in the lost updates in case the leader crashed. The nodes started showing different colors in their application layers because the color depended on the strict sequence of received messages.

3 Evaluation

To solve the problems which persisted in *gms2*. I implemented *gms3* which provided the system with the reliable multicast. Each outgoing message is tagged with a unique reference and sent as an envelope with the payload from the original application message. The sender records the pending send in a map and arms a retry timer. Upon receiving an envelope, the recipient immediately replies with acknowledgment message — regardless of whether the message content is new — before deciding whether to process it. If the sender’s timer fires before the corresponding acknowledgment has been received, it resends the same message under the same reference and decrements a retry counter; after a fixed number of attempts, the recipient is presumed unreachable and the send is abandoned.

Because retries can cause a message to be delivered later than a subsequent message sent for real in the meantime, a slave can no longer assume its messages will always arrive in strict numeric order. Rather than requiring the sender to preserve send order the slave tolerates out-of-order arrival and reconstructs the correct order itself. Each slave keeps a small map, buffer, from sequence number to message content. When an incoming message’s sequence number is greater than the next expected number, it is not discarded and not applied; it is simply stored in buffer under its own number, to be applied once its turn comes.

The *gms3* solves the reliability issues of the previous implementations but at the same time still misses some important aspects. One of the problems is that the failure of slave nodes is not considered in any way and if some of them die the leader continues broadcasting the messages to the failed nodes. The rest of the issues are mentioned in the next section.

4 Conclusions

Summing up the findings of this laboratory work, I can claim that I managed to successfully implement the reliable ordered multicast via the group membership service. While the final version of the GMS seems to solve all of the issues encountered, there are still certain concerns. First of all, the erlang-monitor tool does not guarantee correct failure detection in the distributed environment - the node might delay its response but will be considered as ‘incorrect’. To remedy the risk of losing the leader in this way, I suggest that the new leader election should happen in a quorum, so that the nodes conclude the presence of a new single leader. Secondly, the leader might update its application layer state and then fail before sending the message to the rest of the group. To avoid such situations, it is necessary to guaranty uniform agreement by updating the state of the application layer if and only if at least one ‘correct’ node delivered the message.